Implementing and Tuning Gradient-Based Optimizers

Jeonghoon Park
UNIST
hoonably@unist.ac.kr

1 How the Optimizers Differ from Vanilla Gradient Descent

Vanilla gradient descent updates the parameter vector using only the current gradient,

$$x_{t+1} = x_t - \eta g_t.$$

It is simple, but it has no memory and uses one global step size.

Momentum. Momentum adds a velocity term, so consistent gradient directions accumulate while alternating directions are damped.

RMSProp. RMSProp keeps an exponential moving average of squared gradients and uses it to rescale each coordinate's step size.

Adam. Adam combines the first-moment idea of Momentum with the second-moment scaling idea of RMSProp, with bias correction for the zero initialization of both estimates.

AdamW. AdamW uses the Adam update direction but applies weight decay as a separate shrinkage term instead of mixing it into the adaptive gradient.

The corresponding update rules implemented in the code are

$$\begin{aligned}
\text{Momentum:}\quad & v_t = \beta v_{t-1} + g_t, \quad x_{t+1} = x_t - \eta v_t, \\
\text{RMSProp:}\quad & s_t = \beta s_{t-1} + (1-\beta) g_t^2, \quad x_{t+1} = x_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}, \\
\text{Adam:}\quad & m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\
& \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad x_{t+1} = x_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \\
\text{AdamW:}\quad & d_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad x_{t+1} = x_t - \eta d_t - \eta \lambda x_t.
\end{aligned}$$

Here g_t^2, square roots, and divisions are applied elementwise.

2 Hyperparameter Search Plan and Final Hyperparameters

I tuned the optimizers using visual trajectories together with target-based convergence diagnostics. Since judging trajectories by eye can be subjective, I also recorded when each optimizer first reaches

$$f(x_t) - f^\star \le \tau, \qquad \tau \in \{10^{-2}, 10^{-4}, 10^{-6}\}.$$

The first two thresholds measure practical convergence speed, while 10^{-6} is a stricter final-accuracy check. Because Env 4 has noisy gradients, I interpreted the strictest threshold there more carefully and also checked final loss, final position, and the visual trajectory.

The search was organized as complete candidate settings rather than only changing one optimizer at a time. When a later trial reused an earlier setting, I treated that as a final selection rather than adding a duplicate table row. The final hyperparameters were selected by balancing convergence speed, final loss, final position, stability, and visual behavior.

Trial 1. Momentum was reliable but slow on the plateau environment, so I increased β from .90 to .95. RMSProp was fast in Env 2 and Env 4 but weak on Env 1 and Env 3, so I lowered η to $5\text{e}{-3}$ to

Optimizer	Params.	Env 1	Env 2	Env 3	Env 4
Momentum	$(1e{-}3, .90)$	201 / 407 / 613	968 / 1005 / 1042	364 / 882 / 1432	446 / 551 / 734
	$(1e{-}3, .95)$	**108 / 190 / 272**	520 / 620 / 717	90 / **183** / 392	260 / 282 / 392
	$(1e{-}3, .97)$	157 / 346 / 438	490 / 622 / 779	**35** / 421 / 625	200 / 352 / 547
	$(1e{-}3, .96)$	147 / 234 / 350	472 / 570 / 667	196 / 303 / 416	228 / 278 / 439
	$(1.5e{-}3, .95)$	134 / 212 / 290	**390 / 445 / 549**	170 / 245 / **306**	**188 / 212 / 368**
RMSProp	$(1e{-}2, .99, 1e{-}8)$	1171 / NR / NR	372 / 401 / 427	NR / NR / NR	276 / 388 / 474
	$(5e{-}3, .99, 1e{-}8)$	1504 / NR / NR	945 / 1006 / 1052	1957 / NR / NR	585 / 733 / 824
	$(1e{-}2, .995, 1e{-}8)$	1776 / NR / NR	244 / 267 / **288**	NR / NR / NR	209 / 340 / 448
	$(5e{-}2, .999, 5e{-}2)$	**624 / 1265 / 1751**	**42 / 76** / NR	**202** / NR / NR	**8 / 18 / 243**
	$(1e{-}2, .98, 1e{-}8)$	830 / NR / NR	476 / 506 / 529	NR / NR / NR	296 / 370 / 472
Adam	$(2e{-}2, .90, .999)$	NR / NR / NR	244 / 287 / 329	1749 / NR / NR	226 / 334 / 430
	$(5e{-}2, .90, .99)$	**758 / 917 / 994**	131 / 182 / 220	518 / **638 / 706**	**72 / 90** / 255
	$(4e{-}2, .90, .99)$	826 / 987 / 1064	155 / 201 / 241	561 / 684 / 752	95 / 122 / 237
	$(5e{-}2, .85, .99)$	764 / 928 / 1010	**117 / 141 / 171**	**500** / 699 / NR	81 / 111 / 216
	$(5e{-}2, .90, .995)$	1124 / 1443 / 1609	137 / 180 / 219	695 / 915 / 1053	74 / 93 / **209**
AdamW	$(2e{-}2, .90, .999, 5e{-}2)$	1409 / NR / NR	220 / 264 / 306	NR / NR / NR	207 / 308 / 412
	$(5e{-}2, .90, .99, 1e{-}2)$	**710 / 894 / 978**	**129 / 179 / 218**	529 / NR / NR	**71 / 88** / 214
	$(5e{-}2, .90, .99, 5e{-}4)$	756 / 916 / 993	131 / 182 / 220	518 / 642 / NR	72 / 89 / 254
	$(5e{-}2, .90, .99, 1e{-}4)$	758 / 917 / 994	131 / 182 / 220	**518 / 639 / 877**	72 / 90 / 255
	$(5e{-}2, .90, .995, 1e{-}4)$	1123 / 1442 / 1608	137 / 180 / 219	696 / 918 / 1066	74 / 93 / **208**

Table 1: Target-reaching iterations for each optimizer setting. Each cell reports iter@10^{-2} / iter@10^{-4} / iter@10^{-6}, averaged over seeds 0–99 after excluding the slowest seed. NR means fewer than 99 seeds reached the target. Bold values mark the fastest setting within each optimizer/environment/threshold, and gray rows mark the selected settings.

test whether it was overshooting. Adam and AdamW both used a larger learning rate and smaller β_2 to respond faster; for AdamW I also reduced weight decay because shrinkage can oppose the Rosenbrock optimum at $(1, 1)$.

Trial 2. Because Trial 1 improved Momentum, I tried a larger $\beta = .97$ to see if more accumulation would help the plateau and noisy saddle. For RMSProp, the lower learning rate was too conservative, so I restored $\eta = 1e{-}2$ and raised β to .995 for a smoother squared-gradient estimate. For Adam I reduced β_1 to test less first-moment inertia, while AdamW used $\lambda = 0$ as a diagnostic for whether weight decay caused its Env 3 weakness.

Trial 3. Trial 3 focused on reducing RMSProp's NR entries. A larger RMSProp learning rate with $\beta = .999$ improved Env 1 and the loose Env 3 target, but it also caused overshoot near the optimum in Env 2 and Env 4, so I increased ϵ to $5e{-}2$ to damp overly large effective steps. I also tested an intermediate Momentum value, $\beta = .96$, and restored Adam/AdamW to $\beta_1 = .90$ while trying the smoother $\beta_2 = .995$.

Trial 4. After checking the Trial 3 figures, I compared smaller-step RMSProp candidates because its Rosenbrock path was unstable. The smoother RMSProp candidate helped Env 3 visually but sacrificed too much speed in the other environments, so I kept the aggressive Trial 3 setting and treated Rosenbrock as its limitation. For the final settings, I used $(1.5e{-}3, .95)$ for Momentum, kept Adam's Trial 1 setting after $\eta = 4e{-}2$ was slower, and chose a small nonzero AdamW decay, $\lambda = 10^{-4}$, so AdamW remained distinct from Adam without losing target-reaching behavior.

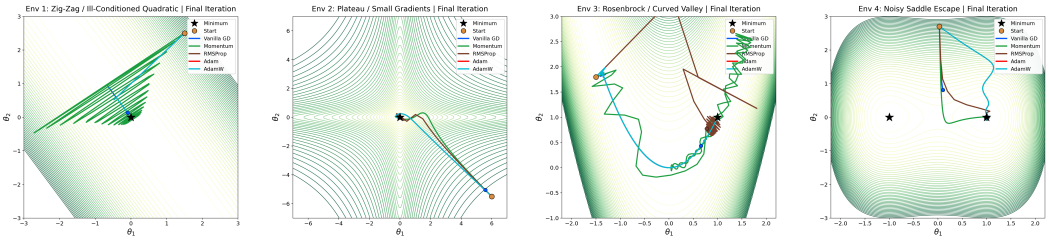
Optimizer	Env 1	Env 2	Env 3	Env 4
Vanilla GD	$1.29e{-}2$ / $1.61e{-}1$	9.89 / 7.53	$1.17e{-}1$ / $6.62e{-}1$	$5.75e{-}1$ / 1.21
Momentum	$4.09e{-}42$ / $1.49e{-}22$	$8.85e{-}22$ / $1.34e{-}11$	$4.98e{-}30$ / $3.14e{-}16$	$7.80e{-}6$ / $3.01e{-}3$
RMSProp	$5.84e{-}8$ / $3.42e{-}4$	$3.96e{-}3$ / $2.83e{-}2$	2.64 / $3.76e{-}1$	$3.24e{-}4$ / $1.82e{-}2$
Adam	$1.40e{-}4$ / $8.49e{-}4$	$1.58e{-}10$ / $5.65e{-}6$	$4.30e{-}3$ / $3.62e{-}3$	$5.17e{-}4$ / $2.56e{-}2$
AdamW	$1.42e{-}3$ / $2.70e{-}3$	$7.56e{-}11$ / $3.91e{-}6$	$2.06e{-}3$ / $2.79e{-}3$	$5.17e{-}4$ / $2.56e{-}2$

Table 2: Endpoint quality averaged over seeds 0–99. Each cell reports final loss / distance to the nearest optimum; smaller is better.

Table 2 is only an endpoint check: Momentum often ends closest after the full budget, while Adam, AdamW, and RMSProp can reach useful targets earlier.

3 Final-Iteration Figures and Environment Comparison

Figure 1 shows the final-iteration figures from `plots/final_figures`. I used these figures together with Tables 1 and 2 to check whether target-reaching speed, final quality, and trajectory shape told the same story.



(a) Env 1: Zig-Zag (b) Env 2: Plateau (c) Env 3: Rosenbrock (d) Env 4: Noisy saddle

Figure 1: Final-iteration trajectories using the selected hyperparameters.

Env 1: Zig-Zag. Momentum gives the cleanest result because the accumulated velocity follows the long direction of the quadratic while damping zig-zag motion. Adam and AdamW approach the minimum but are slower on strict targets, and RMSProp needs the aggressive setting to reach all targets. Vanilla GD is slow because one learning rate must remain stable in the steep direction, which limits movement along the flatter direction.

Env 2: Plateau. Adam and AdamW are strongest if convergence speed is measured by the target iterations. Their adaptive second-moment scaling keeps the effective step size useful in the small-gradient region, while vanilla GD barely moves across the plateau. Momentum eventually ends closest, and RMSProp is fast for loose and medium targets but less reliable at the strictest threshold.

Env 3: Rosenbrock. Momentum reaches all three targets fastest and ends essentially at $(1, 1)$. Adam and AdamW follow the curved valley more effectively than vanilla GD, which remains far from the minimizer. The small AdamW decay slows the strict target, consistent with shrinkage slightly opposing the optimum at $(1, 1)$. RMSProp is weakest here because the aggressive setting reaches only the loose target and does not follow the curved valley well; its selected setting is mainly justified by stronger behavior in the other environments.

Env 4: Noisy Saddle Escape. RMSProp reaches the target thresholds fastest, which suggests that its coordinate-wise scaling moves quickly through the noisy saddle region, while vanilla GD remains far from either minimum. Momentum is slower on the strict threshold but has the best endpoint quality, showing a speed-versus-stability tradeoff under noisy gradients.

4 Which Optimizer Worked Best in Each Environment and Why

The best optimizer depends on whether speed or final precision is emphasized. I used the target-reaching table as the primary comparison, while using the figures and final quality as stability checks.

Environment	Best optimizer	Reason
Env 1: Zig-Zag	Momentum	Fastest strict convergence and nearly zero final loss.
Env 2: Plateau	Adam / AdamW	Fastest target reaching; Momentum has the best final quality.
Env 3: Rosenbrock	Momentum	Best curved-valley convergence and final position.
Env 4: Noisy saddle	RMSProp	Fastest target reaching; Momentum has better final loss.

Table 3: Summary of the strongest optimizer behavior in each environment.